

# EXPERIMENT – 10

**Name:** K ANUSRI

**Roll Number:** 2520030432

**Section:** 8

**Title** Environment Export and Process Management

**Aim** To retrieve the current directory, display the path, process exit requests, clean up resources, save program state, and verify correct behavior. To parse export syntax, update environment variables, validate variable names, handle existing variables, export variables to child processes, and test functionality.

## Theory

The current working directory of a process can be obtained using the `getcwd()` system call.

Environment variables are name-value pairs maintained by the process environment. The `setenv()` function can be used to create or update an environment variable.

An export command can be parsed by separating the variable name and value around the '=' character. Variable names are validated before they are stored. Existing variables can be updated when the same name is exported again.

A child process created using `fork()` inherits the environment of its parent process. The child can access exported variables using `getenv()`. The `waitpid()` system call allows the parent process to wait for the child to complete.

Before termination, the program can save its state and release resources. The `exit` operation performs cleanup and saves the current directory and variable information into a state file.

## Algorithm

1. Start the program.
2. Retrieve the initial working directory using `getcwd()`.
3. Store the current directory as the initial program state.
4. Validate environment variable names using alphabetic, numeric, and underscore rules.
5. Parse export commands using the `NAME=VALUE` format.
6. Separate the variable name from its value.
7. Create a new environment variable using `setenv()`.
8. If the variable already exists, update its value.
9. Display the exported environment variables
10. Create a child process using `fork()`.
11. Verify that the child process can access exported variables using `getenv()`.
12. Wait for the child process using `waitpid()`.
13. Save the current state to a file.
14. Handle exit requests and clean up resources.
15. Verify the complete functionality through automatic and interactive tests.
16. Stop the program.

## CODE



k\_anusri@DESKTOP-TM20BAC



```
k_anusri@DESKTOP-TM20BAQ:~$
k_anusri@DESKTOP-TM20BAQ:~$ #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_LINE 512
#define MAX_VARS 50
#define NAME_SIZE 64
#define VALUE_SIZE 256

typedef struct
{
    char name[NAME_SIZE];
    char value[VALUE_SIZE];
} EnvVariable;

EnvVariable variables[MAX_VARS];
int var_count = 0;

char current_directory[PATH_MAX];
char saved_directory[PATH_MAX];

FILE *state_file = NULL;

int valid_variable_name(const char *name)
{
    int i;

    if (name == NULL || name[0] == '\\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (i = 1; name[i] != '\\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int find_variable(const char *name)
{
    int i;

    for (i = 0; i < var_count; i++)
    {
        if (strcmp(variables[i].name, name) == 0)
            return i;
    }
}
```



k\_anusri@DESKTOP-TM20BA



```
        return -1;
    }

    void save_state()
    {
        int i;

        state_file = fopen("shell_state.txt", "w");

        if (state_file == NULL)
        {
            perror("Unable to save state");
            return;
        }

        fprintf(state_file, "CURRENT_DIRECTORY=%s\n",
                current_directory);

        fprintf(state_file, "VARIABLE_COUNT=%d\n",
                var_count);

        for (i = 0; i < var_count; i++)
        {
            fprintf(state_file, "%s=%s\n",
                    variables[i].name,
                    variables[i].value);
        }

        fclose(state_file);
        state_file = NULL;

        printf("Program state saved successfully.\n");
    }

    void cleanup()
    {
        printf("\nCleaning up resources...\n");

        if (state_file != NULL)
        {
            fclose(state_file);
            state_file = NULL;
        }

        save_state();

        printf("Resources cleaned up successfully.\n");
    }

    void exit_program()
    {
        printf("\nExit request received.\n");

        if (getcwd(current_directory,
                sizeof(current_directory)) != NULL)
        {

```



k\_anusri@DESKTOP-TM20BAC



```
        printf("Saving current directory: %s\n",
               current_directory);
    }

    cleanup();

    printf("Program terminated successfully.\n");
    exit(0);
}

void show_current_directory()
{
    if (getcwd(current_directory,
               sizeof(current_directory)) != NULL)
    {
        printf("Current Directory: %s\n",
               current_directory);
    }
    else
    {
        perror("getcwd");
    }
}

void display_path()
{
    char path[PATH_MAX];

    if (getcwd(path, sizeof(path)) != NULL)
    {
        printf("Complete Path: %s\n", path);
    }
    else
    {
        perror("getcwd");
    }
}

int export_variable(const char *name,
                   const char *value)
{
    int index;

    if (!valid_variable_name(name))
    {
        printf("Invalid variable name: %s\n",
               name);
        return 0;
    }

    index = find_variable(name);

    if (index >= 0)
    {
        strcpy(variables[index].value, value);
    }
}
```



k\_anusri@DESKTOP-TM20BAC



```
{
    strcpy(variables[index].value, value);
    setenv(name, value, 1);

    printf("Existing variable updated: %s=%s\n",
           name, value);

    return 1;
}

if (var_count >= MAX_VARS)
{
    printf("Variable table is full.\n");
    return 0;
}

strcpy(variables[var_count].name, name);
strcpy(variables[var_count].value, value);

setenv(name, value, 1);

var_count++;

printf("Environment variable exported: %s=%s\n",
       name, value);

return 1;
}

void parse_export(char *command)
{
    char *expression;
    char *equals;
    char name[NAME_SIZE];
    char value[VALUE_SIZE];

    expression = command + 7;

    while (*expression == ' ')
        expression++;

    if (*expression == '\\0')
    {
        printf("Usage: export NAME=VALUE\n");
        return;
    }

    equals = strchr(expression, '=');

    if (equals == NULL)
    {
        printf("Invalid export syntax.\n");
        printf("Correct format: export NAME=VALUE\n");
        return;
    }
}
```



k\_anusri@DESKTOP-TM20BAC



```
    if ((equals - expression) >= NAME_SIZE)
    {
        printf("Variable name is too long.\n");
        return;
    }

    strncpy(name,
            expression,
            equals - expression);

    name[equals - expression] = '\0';

    strcpy(value, equals + 1);

    export_variable(name, value);
}

void display_variables()
{
    int i;

    printf("\nExported Environment Variables\n");
    printf("-----\n");

    if (var_count == 0)
    {
        printf("No variables exported.\n");
        return;
    }

    for (i = 0; i < var_count; i++)
    {
        printf("%s=%s\n",
                variables[i].name,
                variables[i].value);
    }
}

void test_child_process()
{
    pid_t pid;
    int status;

    printf("\nCreating child process...\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {

```



k\_anusri@DESKTOP-TM20BAC



```
    char *value;

    printf("Child Process Started\n");
    printf("Child PID: %d\n", getpid());

    value = getenv("STUDENT");

    if (value != NULL)
    {
        printf("Child received STUDENT=%s\n",
            value);
    }
    else
    {
        printf("STUDENT variable not found in child.\n");
    }

    value = getenv("COLLEGE");

    if (value != NULL)
    {
        printf("Child received COLLEGE=%s\n",
            value);
    }
    else
    {
        printf("COLLEGE variable not found in child.\n");
    }

    printf("Child process completed successfully.\n");
    exit(0);
}
else
{
    printf("Parent Process Waiting...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited normally.\n");
        printf("Child Exit Status: %d\n",
            WEXITSTATUS(status));
    }
}
}

void show_help()
{
    printf("\nBuilt-in Commands\n");
    printf("-----\n");
    printf("pwd          - Display current directory\n");
    printf("path         - Display complete path\n");
    printf("export NAME=VAL - Export environment variable\n");
    printf("env          - Display exported variables\n");
    printf("testchild    - Test child process inheritance\n");
}
```

```
printf("path          - Display complete path\n");
printf("export NAME=VAL - Export environment variable\n");
printf("env            - Display exported variables\n");
printf("testchild       - Test child process inheritance\n");
printf("save           - Save current state\n");
printf("help           - Display commands\n");
printf("exit          - Save state and exit\n");
}

void run_tests()
{
    printf("\n===== \n");
    printf(" EXPORT AND ENVIRONMENT TEST\n");
    printf("===== \n");

    printf("\n1. CURRENT DIRECTORY TEST\n");
    printf("----- \n");
    show_current_directory();

    printf("\n2. PATH DISPLAY TEST\n");
    printf("----- \n");
    display_path();

    printf("\n3. EXPORT VARIABLE TEST\n");
    printf("----- \n");

    export_variable("STUDENT", "Mohith");
    export_variable("COLLEGE", "BTech");

    printf("\n4. EXISTING VARIABLE UPDATE TEST\n");
    printf("----- \n");

    export_variable("STUDENT", "Student");

    printf("\n5. INVALID VARIABLE NAME TEST\n");
    printf("----- \n");

    export_variable("123INVALID", "test");

    printf("\n6. EXPORTED VARIABLE DISPLAY\n");
    printf("----- \n");

    display_variables();

    printf("\n7. CHILD PROCESS EXPORT TEST\n");
    printf("----- \n");

    test_child_process();

    printf("\n8. STATE SAVE TEST\n");
    printf("----- \n");

    save_state();

    printf("\nAll automatic tests completed successfully.\n");
}
```

```
int main()
{
    char line[MAX_LINE];

    printf("=====\n");
    printf(" ENVIRONMENT EXPORT AND PROCESS MANAGEMENT\n");
    printf("=====\n");

    if (getcwd(current_directory,
               sizeof(current_directory)) == NULL)
    {
        perror("getcwd");
        return 1;
    }

    strcpy(saved_directory,
            current_directory);

    printf("\nInitial Working Directory: %s\n",
            current_directory);

    run_tests();

    printf("\n=====\n");
    printf(" INTERACTIVE COMMAND TEST\n");
    printf("=====\n");

    printf("Enter commands below.\n");
    printf("Examples:\n");
    printf("  pwd\n");
    printf("  path\n");
    printf("  export NAME=VALUE\n");
    printf("  env\n");
    printf("  testchild\n");
    printf("  save\n");
    printf("  help\n");
    printf("  exit\n\n");

    while (1)
    {
        printf("export-shell> ");

        if (fgets(line,
                  sizeof(line),
                  stdin) == NULL)
        {
            exit_program();
        }

        line[strcspn(line, "\n")] = '\0';

        if (strlen(line) == 0)
            continue;

        if (strcmp(line, "pwd") == 0)
```

```
    if (strlen(line) == 0)
        continue;

    if (strcmp(line, "pwd") == 0)
        show_current_directory();
    else if (strcmp(line, "path") == 0)
        display_path();
    else if (strncmp(line, "export ", 7) == 0)
        parse_export(line);
    else if (strcmp(line, "env") == 0)
        display_variables();
    else if (strcmp(line, "testchild") == 0)
        test_child_process();
    else if (strcmp(line, "save") == 0)
        save_state();
    else if (strcmp(line, "help") == 0)
        show_help();
    else if (strcmp(line, "exit") == 0)
        exit_program();
    else
        printf("Invalid command: %s\n", line);
}

return 0;
}
```

```
if (strlen(line) == 0)
```

```
    continue;
```

```
if (strcmp(line, "pwd") == 0)
```

```
    show_current_directory();
```

```
else if (strcmp(line, "path") == 0)
```

```
    display_path();
```

```
else if (strncmp(line, "export ", 7) == 0)
```

```
    parse_export(line);
```

```
else if (strcmp(line, "env") == 0)
```

```
    display_variables();
```

```
else if (strcmp(line, "testchild") == 0)
```

```
    test_child_process();
```

```
else if (strcmp(line, "save") == 0)
```

```
    save_state();
```

```
else if (strcmp(line, "help") == 0)
```

```
    show_help();
```

```
else if (strcmp(line, "exit") == 0)
```

```
    exit_program();
```

```
else
```

```
printf("Invalid command: %s\n", line);
```

```
} return 0; }
```

## Execution Screenshots

1: Current directory, path display, export variable, update, validation, and exported variable tests

```
=====
ENVIRONMENT EXPORT AND PROCESS MANAGEMENT
=====

Initial Working Directory: /home/mohith_07

=====
EXPORT AND ENVIRONMENT TEST
=====

1. CURRENT DIRECTORY TEST
-----
Current Directory: /home/mohith_07

2. PATH DISPLAY TEST
-----
Complete Path: /home/mohith_07

3. EXPORT VARIABLE TEST
-----
Environment variable exported: STUDENT=Mohith
Environment variable exported: COLLEGE=BTech

4. EXISTING VARIABLE UPDATE TEST
-----
Existing variable updated: STUDENT=Student

5. INVALID VARIABLE NAME TEST
-----
Invalid variable name: 123INVALID

6. EXPORTED VARIABLE DISPLAY
-----

Exported Environment Variables
-----
STUDENT=Student
COLLEGE=BTech
```

2: Child process export test, state save test, and interactive command section

## 7. CHILD PROCESS EXPORT TEST

---

```
Creating child process...
Parent Process Waiting...
Child Process Started
Child PID: 736
Child received STUDENT=Student
Child received COLLEGE=BTech
Child process completed successfully.
Child exited normally.
Child Exit Status: 0
```

## 8. STATE SAVE TEST

---

```
Program state saved successfully.
```

```
All automatic tests completed successfully.
```

---

### INTERACTIVE COMMAND TEST

---

```
Enter commands below.
```

```
Examples:
```

```
pwd
path
export NAME=VALUE
env
testchild
save
help
exit
```

### Result

The program was executed successfully. The current directory and complete path were retrieved and displayed. Export syntax was parsed, environment variables were created and updated, invalid variable

names were rejected, and exported variables were inherited by the child process. The program state was saved successfully, demonstrating the required environment export, process management, cleanup, and verification functionality.